

Full Stack Development with MERN - Project Documentation

Date: 11 June 2026

1. Introduction

- **Project Title:** MERN Stack E-commerce Application
- **Team Members:** Development Team

2. Project Overview

- **Purpose:** To provide a fully functional, secure, and scalable online storefront for customers and an inventory management platform for administrators.
- **Features:** User Authentication, Product Catalog, Shopping Cart, Checkout flow with Payment Gateway (Planned), Order Tracking, Admin Dashboard, Product Reviews.

3. Architecture

- **Frontend:** Built with React.js using functional components and hooks for state management. Communicates with backend via Axios.
- **Backend:** Node.js and Express.js RESTful API serving JSON responses and handling business logic.
- **Database:** MongoDB (via Mongoose ODM) storing Users, Products, Categories, Orders, and Reviews.

4. Setup Instructions

- **Prerequisites:** Node.js (v14+), MongoDB (Local or Atlas), Payment Gateway Account, Cloundinary Account.
- **Installation:**
 1. Clone the repository.
 2. Run `npm install` in both `/client` and `/server` directories.
 3. Create a `.env` file in the `/server` directory with `MONGO_URI`, `JWT_SECRET`, `PAYMENT_API_KEY`, `CLOUDINARY_URL`.
 4. Optional: Run `npm run seed` to populate initial database data.

5. Folder Structure

- **Client:** Contains `/src/components`, `/src/pages`, `/src/actions`, `/src/reducers`.
- **Server:** Contains `/controllers`, `/models`, `/routes`, `/middleware`, `/config`.

6. Running the Application

- **Frontend:** `npm start` in the client directory (runs on port 3000).
- **Backend:** `npm run dev` in the server directory (runs on port 5000 using Nodemon).

7. API Documentation (Example)

- `POST /api/users/login` - Authenticate user & get token.
- `GET /api/products` - Fetch all products.
- `GET /api/products/:id` - Fetch single product.
- `POST /api/orders` - Create a new order (requires Auth).

8. Authentication

Authentication is handled via JWT (JSON Web Tokens). Upon login, the server signs a token containing the User ID and Role. This token is stored in an HTTP-only cookie to prevent XSS attacks and sent with subsequent requests to protected routes.

9. User Interface

(Screenshots or UI descriptions should be placed here). The UI uses modern CSS frameworks, providing a responsive grid layout for products and a step-by-step wizard for checkout.

10. Testing

Testing strategy includes manual UI testing, Postman API endpoint verification, and potential Jest/React Testing Library unit tests for critical components like the cart reducer.

11. Known Issues

- Cart state might desync if the user logs in from multiple devices simultaneously.

12. Future Enhancements

- Adding AI-driven product recommendations.
- Implementing multi-vendor support.
- Advanced shipping cost calculators based on external logistics APIs.